LE GOUVERNEMENT
DU GRAND-DUCHÉ DE LUXEMBOURG
Ministère de l'Éducation nationale,
de l'Enfance et de la Jeunesse

EXAMEN DE FIN D'ÉTUDES SECONDAIRES – Sessions 2025

QUESTIONNAIRE

Date :			*Horaire :*			*Durée :*	
Discipline :		*Type :*		*Section(s) :*			
					Numéro du candidat :		

Modelling part

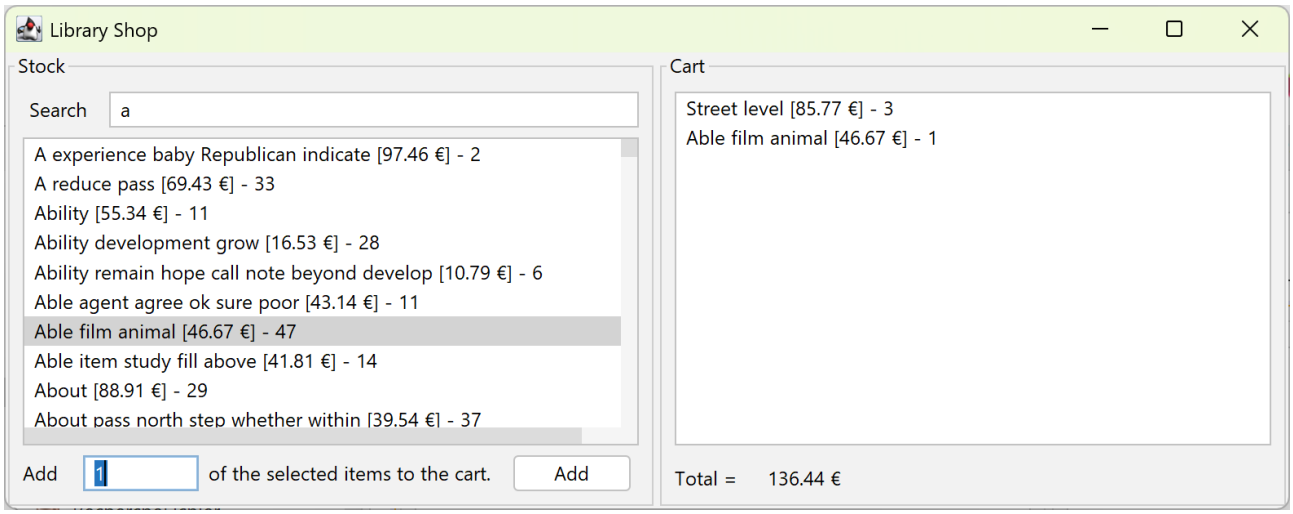
11 points

- The modelling part must be written on paper and must be handed in before the implementation part can start.
- The modelling part is due after 35 minutes at the latest, but you are free to hand it in earlier.

You are required to setup the model of a cash register application (DE: das Kassenprogramm, FR: la caisse enregistreuse) that connects to a stock of items:

- An item has a name as well as a price it will be sold at.
- The stock contains a certain amount of each item.
- When somebody buys some items, the cashier (DE: der Kassierer / FR: le caissier) puts the item as well as the amount of that item the client buys into the cart.
    - Items that figure in the cart will be removed from the stock.
    - The maximum number of items in the cart is the number of items in the stock.

The application might look like this:



On the left side, items from the stock can be searched by their name.
Some controls allow to move an item from the stock into the clients cart.
On the right side the items in the cart are listed as well as the number total value of the cart.

Use the principles of object-oriented programming to create class diagram of the application model on paper, using the usual UML conventions.

Adding comments onto your diagram is recommended in order to explain your choice!

- Add the necessary classes to model the items, the stock and the cart.
- First, limit yourself to the basic attributes of these classes!
- Add all nessessary methods to cover the following use-cases:
    a. adding items to the stock,
    b. putting something from the stock into the cart,
    c. removing an item from the cart back into the stock,
    d. displaying the stock or cart as JList,
    e. displaying the total value of the cart.
- Add methods allowing to:
    a. save / load the stock to / from a file,
    b. save / load the cart to / from a file,
    c. search the stock for a given item by it's name,
- As you can see on the screenshot, the items in the stock are ordered by their names, while those in the cart are not. Make a proposition on how to integrate this into your model. Give any necessary detail to fully understand your idea and argument your choice!

In your working directory (to be defined by each school), you will find a folder called EXAMEN_CI. Rename this folder by replacing the current name with your exam code (example of notation: LXY_CI2_05). All your files should be saved in this folder, which will be called "your folder" afterwards.

Implementation part

49 points

Next, you will code the cash register application as it has already been described in first part of the exam. The skeleton of this program is given in your folder. Open it and complete it according to the following instructions.

All exceptions must be handled in the main `MainFrame`, i.e. all possible exceptions are passed on to the calling class.

The "ItemNotFoundException" class

1 point

This class represents an exception which is to be thrown when the application does not find an item. It inherits from the `Exception` class. Its unique constructor gets a string as parameter. **1 point**

The "Item" class

2.5 points

This class represents an item which can be either held inside the stock or the cart. It has the following attributes, which have default getters and setters: **1 point**

- `name` the name of the item
- `price` the price the item is sold at

Besides the constructor, which initialises all attributes, this class has the following method: **1 point**

- The method `toString()` returns a string representation of the item and has the following format: **0.5 points**

`<name> [<price>]`

The "ItemNode" class

4 points

This class represents a node of a linked list containing items. It has the following attributes: **1 point**

- `item` the item it holds
- `count` the number of items
- `next` a pointer to the next `ItemNode` in the list

Besides the constructor, which initialises a new `ItemNode`, and all the standard getters and setters for its' attributes, the class implements the Comparable interface and has the following methods: **1.5 point**

- The method `compareTo(…)` compares the names of the item with the name of the items passed as parameter. **1 points**
- The method `toString()` returns a string representation of the item and has the following format: **0.5 points**

`<name> [<price>] – <count>`

The "ItemList" class ***26.5 points***

This class, which holds a list of items, represents either the stock or a cart. I has the following attributes: **1 point**

- `root` a pointer to the first item in the list
- `size` the actual number of items stored in the list

You will need to implement the following methods:

- The method `addOrIncrementCount(…)` modifies the `ItemList` like this: The first parameter indicates the item that may be added to the list. The second parameter indicates the quantity of that item to be added to the list. If the item is already present in the list, it's current count has to be incremented. If not the item and its count (=amount of items) needs to be added to the end of the list. This method returns the amount of that item, passed as parameter, contained in the list after the operation. **6 points**
- The method `get(…)` calls the method `getRecursive(…)` which will recursively pass through the list in order to retrieve an item at a given position. If the position is not valid, an `IndexOutOfBoundsException` has to be thrown! **4.5 points**
- The method `removeOrDecrement(…)` decrements the number of an item in the list or removes the it completely in case the number of items in the list would become zero or less.
  - The method returns the effective number of items removed.
  - The method throws an `ItemNotFoundException` in case the item to be removed is not contained in the list.

  Example: If you want to remove 7 given items, but the list only holds 5 of them, the method will return 5! **8 points**
- The method `saveToCsv(…)` saves the actual list into a CSV file. Each line contains the information about a single item and has the following format: **2 points**

  `<name>,<price>,<count>`
- The method `loadFromCsv(…)` loads a list from the CSV file stored in the above described format. **2.5 points**
- The method `toArray()` returns the list in form of a static array of `ItemNode` so that it can be displayed in an `JList`. **2.5 points**

The "Cart" class ***2.5 points***

This class extends the `ItemList` class by implementing the following specialized method: **0.5 point**

- The method `getTotal(…)` calculates and returns the total price of the items in the list. **2 points**

The "Stock" class ***3.5 points***

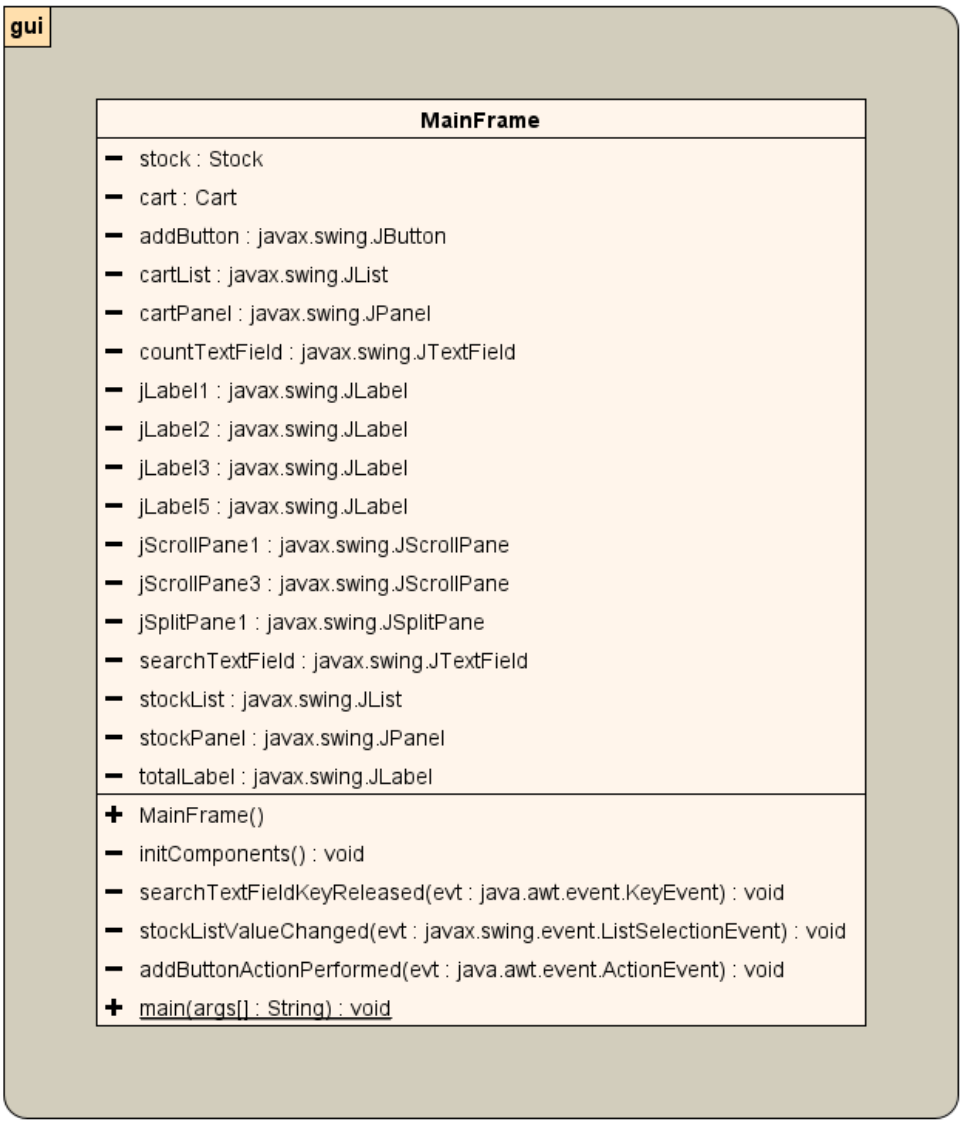
This class extends the `ItemList` class by implementing the following specialized method: **0.5 point**

- The method `searchStartWith()` returns a list of all items whose names start with the prefix passed as parameter. The items inside the returned list need to be sorted alphabetically by their name. **3 points**

The "MainFrame" class ***9 points***

The graphical interface is provided and already contains source code that you need to complete following the instructions below:

- All exceptions the model may rise must be caught and displayed to the user using the `JOptionPane.showMessageDialog(…)` method. **1.5 points**
- The class has two attributes of the type `Stock` and `Cart`. The first one represents the stock, the second one the client's cart. The stock has to be loaded from the file "books_stock.csv" and all items must be displayed. **1 point**
- Each time a key has been released within the search field, only items whose name start with the given search string are being displayed in the stock list. **1.5 point**
- When the user hits the "Add" button, the indicated number of items are being transferred from the stock to the cart. If more items are to be transferred than are in stock, then only the number of items present in the stock can be transferred to the cart. After the transfer, the stock list as well as the cart list and its total has to be updated. The selected item in the stock needs to stay selected! (Please check the demo application …) **5 points**



UML of the model